A2:2017 – Broken Authentication
A3:2017 – Sensitive Data Exposure
A4:2017 – XML External Entities (XXE)
A5:2017 – Broken Access Control
A6:2017 – Security Misconfiguration
A7:2017 – Cross-Site Scripting (XSS)
A8:2017 – Insecure Deserialization
A9:2017 – Using Components with Known Vulnerabilities
A10:2017 – Insufficient Logging & Monitoring
While this book was being written, OWASP published a dedicated API Security Top 10, which you can find on their website: https://owasp.org/www-project-api-security/
It's important to note that although every vulnerability in the Top 10 is worth learning about, avoiding the Top 10 will not in itself make your application secure. There is no simple checklist of vulnerabilities to avoid. Instead, this book will teach you the general principles to avoid entire classes of vulnerabilities.

An *injection attack* can occur anywhere that you execute dynamic code in response to user input, such as SQL and LDAP queries, and when running operating system commands.

> **DEFINITION** An ***injection attack*** occurs when unvalidated user input is included directly in a dynamic command or query that is executed by the application, allowing an attacker to control the code that is executed.

If you implement your API in a dynamic language, your language may have a built-in `eval()` function to evaluate a string as code, and passing unvalidated user input into such a function would be a very dangerous thing to do, as it may allow the user to execute arbitrary code with the full permissions of your application. But there are many cases in which you are evaluating code that may not be as obvious as calling an explicit `eval` function, such as